63-FOC 16

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。上节课迭代方程还有一些没有讲完，今天我们接着讲。

```
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6,A,[w9],w6
    msc         w5*w6,B,[w8]+=2,w4
    add         w4,ACCUM_ANGLE,ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6,A,[w9],w6
    mac         w5*w6,B,[w8]+=2,w4
    subbr       w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
    lsr         w5,w5
    mov         ACCUM_ANGLE,w0
```

我们继续逆时针旋转部分，逆时针的话就是跳转到红色箭头的部分

$$x^{(i+1)} = x^{(i)} - y^{(i)} * 2^{-i}$$
$$y^{(i+1)} = y^{(i)} + x^{(i)} * 2^{-i}$$
$$z^{(i+1)} = z^{(i)} - \theta^{(i)}$$

此时，对应使用的迭代方程就是这个， 第一个方程，是计算一个减法。

```
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6,A,[w9],w6
    mac         w5*w6,B,[w8]+=2,w4
    subbr       w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个红框实现了第一行的迭代方程。

MSC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,AWB	相乘并从累加器中减去乘积	1	1	301

语法： { 标号 :} MSC Wm * Wn, Acc {,[Wx], Wxd} {,[Wy], Wyd} {,AWB}
{,[Wx] + = kx, Wxd} {,[Wy] + = ky, Wyd}
{,[Wx] – = kx, Wxd} {,[Wy] – = ky, Wyd}
{,[W9 + W12], Wxd} {,[W11 + W12],Wyd}

操作数：
Wm * Wn ∈ [W4 * W5, W4 * W6, W4 * W7, W5 * W6, W5 * W7, W6 * W7]
Acc ∈ [A,B]
Wx ∈ [W8, W9]； kx ∈ [-6, -4, -2, 2, 4, 6]； Wxd ∈ [W4 ... W7]
Wy ∈ [W10, W11]； ky ∈ [-6, -4, -2, 2, 4, 6]； Wyd ∈ [W4 ... W7]
AWB ∈ [W13, [W13] + = 2]

操作：
(Acc(A 或 B)) – (Wm) * (Wn) → Acc(A 或 B)
([Wx]) → Wxd； (Wx) + kx → Wx
([Wy]) → Wyd； (Wy) + ky → Wy
舍入后的 (Acc(B 或 A)) → AWB

受影响的状态位： OA、 OB、 OAB、 SA、 SB 和 SAB

初始时刻的 x 是存储在累加器 A 中的，而 y 是存储在 w6 中的，2^（-i）是存在 w5 寄存器中的，所以，w5*w6 对应的就是 2^(-i)*y，所以指令执行后，也就是实现 x-2^(-i)*y。

大家通过这条指令可以体会到这个单片机功能的强大，一般单片机是没有这么功能强大的指令的。这样复杂的计算，只需 1 个机器周期就可以实现。体现了单片机设计的专业性。

```
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
```

然后，这个红框的地方是一个预取的功能。

$$x^{(i+1)} = x^{(i)} - y^{(i)} * 2^{-i}$$
$$y^{(i+1)} = y^{(i)} + x^{(i)} * 2^{-i}$$
$$z^{(i+1)} = z^{(i)} - \theta^{(i)}$$

因为我们知道，下面的迭代方程计算，会用到 x 值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    do          #9,EndCordic
    sac.r       A,[w9]
    sac.r       B,w6
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6,A,[w9],w6
    msc         w5*w6,B,[w8]+=2,w4
    add         w4,ACCUM_ANGLE,ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
```

而 x 的值呢，在每次迭代开始前，都会从累加器中取出来，存到堆栈临时空间中。

```
rotate_neg: ;逆时针旋转
    msc         w5*w6,A,[w9],w6
    mac         w5*w6,B,[w8]+=2,w4
    subbr       w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
```

所以，这句话的作用，就是把下面要用到的 x 取出来，为下面迭代做准备。而这里的预取功能，实际上 w9 就派上用场了。

语法：	{标号:} MSC	Wm * Wn, Acc	{,[Wx], Wxd}	{,[Wy], Wyd} {,AWB}
			{,[Wx] + = kx, Wxd}	{,[Wy] + = ky, Wyd}
			{,[Wx] – = kx, Wxd}	{,[Wy] – = ky, Wyd}
			{,[W9 + W12], Wxd}	{,[W11 + W12],Wyd}
操作数：	Wm * Wn ∈ [W4 * W5, W4 * W6, W4 * W7, W5 * W6, W5 * W7, W6 * W7] Acc ∈ [A,B] Wx ∈ [W8, W9]； kx ∈ [-6, -4, -2, 2, 4, 6]； Wxd ∈ [W4 ... W7] Wy ∈ [W10, W11]； ky ∈ [-6, -4, -2, 2, 4, 6]； Wyd ∈ [W4 ... W7] AWB ∈ [W13, [W13] + = 2]			
操作：	(Acc(A 或 B)) – (Wm) * (Wn) → Acc(A 或 B) ([Wx]) → Wxd； (Wx) + kx → Wx ([Wy]) → Wyd； (Wy) + ky → Wy 舍入后的 (Acc(B 或 A)) → AWB			
受影响的状态位：	OA、OB、OAB、SA、SB 和 SAB			

大家看下，预取这里使用到的 W 寄存器是有要求的，Wx 只能选择 W8 或者 W9。Wxd 只能在 W4~W7 中选择。

```
rotate_neg: ;逆时针旋转
    msc         w5*w6,A,[w9],w6
    mac         w5*w6,B,[w8]+=2,w4
    subbr       w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
```

所以，我们之前把 W14 的数据传送给 W9，在这里就用上了。就是为了这里预取方便。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
CordicRun:
    mov     w14,w9
    clr     ACCUM_ANGLE
QuadAdjust:
;如果输入的点(x,y)在第二象限或者在第三象限，进行象限调整 第二象限调
    cp0     w1
    bra     NN,CordicSetup
    mov     w1,[w9]
    cp0     w0
    bra     LE,QuadIIIQuadAdjust
    mov     w0,w1
    neg     [w9],w0
    mov     #PI_DIV_2,ACCUM_ANGLE  ;第二象限变到第一象限，顺时
    bra     CordicSetup
QuadIIIQuadAdjust:
    neg     w0,w1
    mov     [w9],w0
    mov     #NEG_PI_DIV_2,ACCUM_ANGLE ;第三象限变到第四象限，逆
CordicSetup:
    lac     w0,#1,B
    lac     w1,#1,A
    movpag  #psvpage(_Cordic_Table),DSRPAG
    mov     #psvoffset(_Cordic_Table),w8
    mov     #0x7FFF,w5
    do      #9,EndCordic
    sac.r   A,[w9]
    sac.r   B,w6
    cp0     w6
    bra     N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
    mac     w5*w6,A,[w9],w6
    msc     w5*w6,B,[w8]+=2,w4
    add     w4,ACCUM_ANGLE,ACCUM_ANGLE
    bra     EndCordic
rotate_neg: ;逆时针旋转
    msc     w5*w6,A,[w9],w6
    mac     w5*w6,B,[w8]+=2,w4
    subbr   w4,ACCUM_ANGLE,ACCUM_ANGLE
```

这就是开始将 W14 内容传递给 W9 的原因。

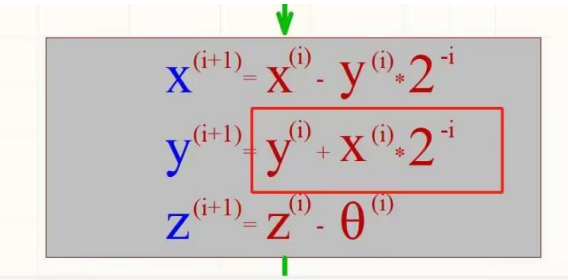
```
rotate_neg: ;逆时针旋转
    msc     w5*w6,A,[w9],w6
    mac     w5*w6,B,[w8]+=2,w4
    subbr   w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
```

那么这样的话，此时 w6 寄存器中，就存储了 x 的值。

```
rotate_neg: ;逆时针旋转
    msc     w5*w6,A,[w9],w6
    mac     w5*w6,B,[w8]+=2,w4
    subbr   w4,ACCUM_ANGLE,ACCUM_ANGLE
EndCordic:
```

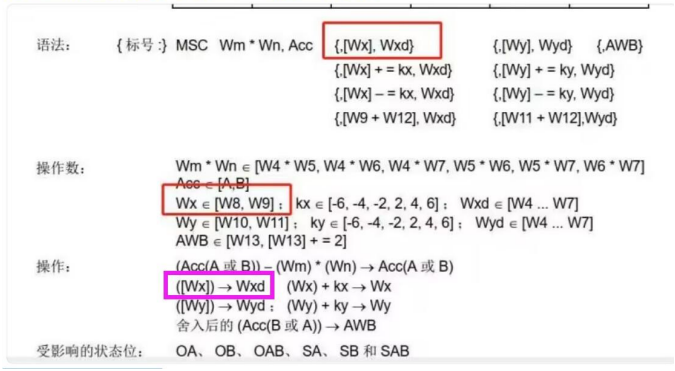
那么，接着呢，就计算第二个迭代方程。因为 B 中存储的是 y 的值，w5 还是 2^(-i)，w6 此时是 x。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



这样的话，使用 mac 指令，就可以计算 y+x*2^(-i)。

问：这里是预取的 w9 再传递给 w6 吗？
答：是的，是 W9 中地址里数据，传递给 W6。看下面粉红色。



MAC	Wm*Wn,Acc,[Wx],Wxd,[Wy],Wyd,AWB	相乘并累加	1	1	275

mac 指令，跟 msc 是类似的，mac 把乘积加到累加器，msc 是从累加器中减去乘积，指令使用的方法都是一样的。

```
    add     w4, ACCUM_ANGLE, ACCUM_ANGLE
    bra     EndCordic
rotate_neg: ;逆时针旋转
    msc     w5*w6, A, [w9], w6
    mac     w5*w6, B, [w8]+=2, w4
    subbr   w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
```

因为 mac 预取的指令，对应的 Wx 寄存器要么是 w8，要么是 w9，所以表的首地址就存到 w8 中了。

```
    add     w4, ACCUM_ANGLE, ACCUM_ANGLE
    bra     EndCordic
rotate_neg: ;逆时针旋转
    msc     w5*w6, A, [w9], w6
    mac     w5*w6, B, [w8]+=2, w4
    subbr   w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
```

所以，这里就查表取出角度，放在 w4 寄存器中了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

语法： {标号:} MAC Wm*Wn, Acc {,[Wx], Wxd} {,[Wy],Wyd} {,AWB}
{,[Wx] + = kx, Wxd} {,[Wy] + = ky, Wyd}
{,[Wx] – = kx, Wxd} {,[Wy] – = ky, Wyd}
{,[W9 + W12],Wxd} {,[W11 + W12],Wyd}

操作数： Wm * Wn ∈ [W4 * W5, W4 * W6, W4 * W7, W5 * W6, W5 * W7, W6 * W7]
Acc ∈ [A,B]
Wx ∈ [W8, W9]； kx ∈ [-6, -4, -2, 2, 4, 6]； Wxd ∈ [W4 ... W7]
Wy ∈ [W10, W11]； ky ∈ [-6, -4, -2, 2, 4, 6]； Wyd ∈ [W4 ... W7]
AWB ∈ [W13, [W13] + = 2]

操作： (Acc(A 或 B)) + (Wm) * (Wn) → Acc(A 或 B)
([Wx]) → Wxd； (Wx) + kx → Wx
([Wy]) → Wyd； (Wy) + ky → Wy
舍入后的 (Acc(B 或 A)) → AWB

受影响的状态位： OA、OB、OAB、SA、SB 和 SAB

可以看到 mac 指令和 msc 类似，只不过一个是减，一个是加。

```
    add         w4, ACCUM_ANGLE, ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
```

而后面的加 2，是指的 w8 中存储的地址数据加 2， 而后面的加 2，是指的 w8 中存储的地址数据加 2，这也是为下一次查表做准备。

```
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
```

然后呢，因为是逆时针旋转，所以就使用累加器中原始的角度值减去本次旋转的角度。

SUBBR	Wb,Ws,Wd	Wd = Ws – Wb – (C̄)	1	1(5)	420

注意看一下被减数和减数。

$$x^{(i+1)} = x^{(i)} - y^{(i)} * 2^{-i}$$

$$y^{(i+1)} = y^{(i)} + x^{(i)} * 2^{-i}$$

$$z^{(i+1)} = z^{(i)} - \theta^{(i)}$$

这是逆时针对应的角度累加器计算方程。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
```

所以，这样的话，一次迭代就计算完毕了。

```
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
    lsr         w5, w5
    mov         ACCUM_ANGLE, w0
```

迭代方程计算完之后，要更新 2^(-i)的值。我们只需要把 w5 的值右移一位，就相当于 i 增大 1， 这样的话，一次迭代计算才算完成。

我们再来看一下循环次数。

```
    mov         #0x7FFF, w5
    do          #9, EndCordic
    sac.r       A, [w9]
    sac.r       B, w6
    cp0         w6
    bra         N, rotate_neg ;y<0，逆时针旋转，y>0，顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6, A, [w9], w6
    msc         w5*w6, B, [w8]+=2, w4
    add         w4, ACCUM_ANGLE, ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
    lsr         w5, w5
    mov         ACCUM_ANGLE, w0
```

因为我们采用的是 do 循环指令，循环的次数是 9+1=10 次。也就是说，我们设定的迭代次数是 10。所以，只要迭代次数没有结束，就又会循环执行循环体中的内容。此时呢，就把 x 跟 y 的值取出来，分别存放到[w9]跟 w6 中，为后面再次迭代计算做准备。

```
    sac.r       A, [w9]
    sac.r       B, w6
    cp0         w6
    bra         N, rotate_neg ;y<0，逆时针旋转，y>0，顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6, A, [w9], w6
    msc         w5*w6, B, [w8]+=2, w4
    add         w4, ACCUM_ANGLE, ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
```

接着呢，又会判断 y 的正负。如果 y 还是负的，则继续逆时针旋转，还会再次计算逆时针的迭代方程。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

而如果 y 不是负的，则说明要顺时针旋转。

```
    cp0         w6
    bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6,A,[w9],w6
    msc         w5*w6,B,[w8]+=2,w4
    add         w4,ACCUM_ANGLE,ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6,A,[w9],w6
```

顺时针的话，对应的就是这段代码。

$$x^{(i+1)} = x^{(i)} + y^{(i)} * 2^{-i}$$
$$y^{(i+1)} = y^{(i)} - x^{(i)} * 2^{-i}$$
$$z^{(i+1)} = z^{(i)} + \theta^{(i)}$$

先计算一个累加，后计算一个累减，然后再计算角度累加，使用的指令呢，还是 mac 跟 msc 。

```
14      cp0         w6
15      bra         N,rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
16  rotate_pos: ;顺时针旋转
17      mac         w5*w6,A,[w9],w6
18      msc         w5*w6,B,[w8]+=2,w4
19      add         w4,ACCUM_ANGLE,ACCUM_ANGLE
20      bra         EndCordic
21  rotate_neg: ;逆时针旋转
22      msc         w5*w6,A,[w9],w6
23      mac         w5*w6,B,[w8]+=2,w4
24      subbr       w4,ACCUM_ANGLE,ACCUM_ANGLE
25  EndCordic:
26      lsr         w5,w5
27      mov         ACCUM_ANGLE,w0
28  ExitCordic:
```

计算完成之后，仍然会更新 2^(-i)，然后，如果旋转次数还没有到，则会继续循环。
有关循环次数可以查看 do 指令。

例 1：

```
002000 LOOP6:   DO    #5, END6    ; 启动 DO 循环（执行循环 6 次）
002004          ADD   W1, W2, W3 ; 循环中第一条指令
002006          . . .
002008          . . .
00200A END6:    SUB   W2, W3, W4 ; 循环中最后一条指令
00200C          . . .
```

lsr W5,W5 是循环体中的最后一条指令。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    do          #9, EndCordic
    sac.r       A, [w9]
    sac.r       B, w6
    cp0         w6
    bra         N, rotate_neg ;y<0，逆时针旋转，y>0,顺时针旋转
rotate_pos: ;顺时针旋转
    mac         w5*w6, A, [w9], w6
    msc         w5*w6, B, [w8]+=2, w4
    add         w4, ACCUM_ANGLE, ACCUM_ANGLE
    bra         EndCordic
rotate_neg: ;逆时针旋转
    msc         w5*w6, A, [w9], w6
    mac         w5*w6, B, [w8]+=2, w4
    subbr       w4, ACCUM_ANGLE, ACCUM_ANGLE
EndCordic:
    lsr         w5, w5
    mov         ACCUM_ANGLE, w0
ExitCordic:
```

等到迭代次数到了之后，就会执行红框中的 mov 指令，这句话呢，就是把角度累加器中的角度值传递给 w0。而这也就完成了函数对应的功能：求解角度。

```
EndCordic:
    lsr         w5, w5
    mov         ACCUM_ANGLE, w0
ExitCordic:
    pop         DSRPAG
    pop         CORCON
    ulnk
    return
/*=============================================
```

结束之后呢，就会恢复现场，加上 return，这个子函数就结束了。前面内容，大家有疑问么？

问：“而后面的加 2，是指的 w8 中存储的地址数据加 2，这也是为下一次查表做准备”，问为什么是 2？

答：因为一个角度数据 16 位，占两个地址空间。

```
/*=====================================
_Cordic_Table:
    .word   0x2000  ;45度
    .word   0x12E4  ;26.5650511771度
    .word   0x09FB  ;14.0362434679度
    .word   0x0511  ;7.1250163489度
    .word   0x028B  ;3.5763343750度
    .word   0x0146  ;1.7899106082度
    .word   0x00A3  ;0.8951737102度
    .word   0x0051  ;0.4476141709度
    .word   0x0029  ;0.2238105004度
    .word   0x0014  ;0.1119056771度
    .word   0x000A  ;0.0559528919度
    .word   0x0005  ;0.0279764526度
    .word   0x0003  ;0.0139882271度
/*=====================================
```

图 4-6：程序存储器构成



只使用低 16 位，高 8 位是空着的，右边地址递增 2。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

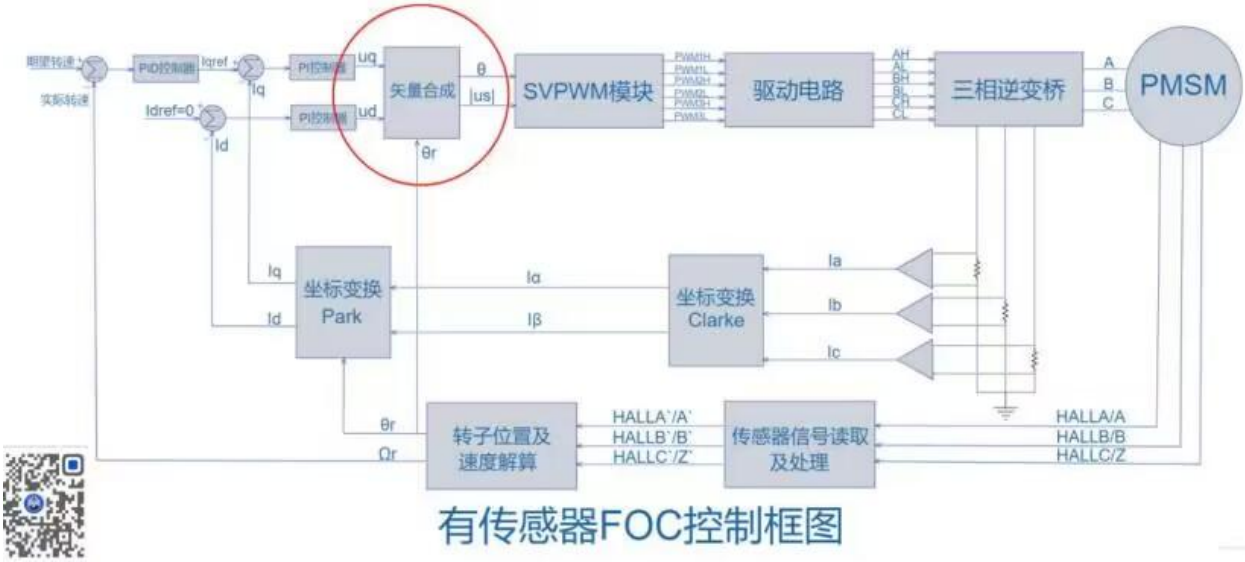
```
extern unsigned int CalSqrt(unsigned int x);
extern signed int CalAtan(signed int y,signed int x);
```

然后，我们在 C 文件中加上调用声明，就可以使用这个函数了。

```
if(tFoc.Direction == CCW)
    tUm.qDQAngle = CalAtan(-tUm.qUd,tUm.qUq);
else
    tUm.qDQAngle = CalAtan(-tUm.qUd,-tUm.qUq);
```

这样的话，就可以求解出超前角了。这个 CORDIC 算法在有的单片机中是可以硬件实现的，这也是目前的主流做法，但并不是所有单片机都能支持这么做。有的单片机中可能会提供库函数来实现，我们也可以封成库函数调用，但我们课程并没有这么做，而是尝试给大家讲解。大家也可以 C 语言试一下，单片机不一定能行，当然如果单片机速度很快则不用担心。如果产品对性价比有追求的话，还是尽量用汇编。

问：那浮点型做输入是要先转换成定点吗？
答：不一定，两种做法都有人在使用。



有传感器FOC控制框图

好了，现在我们来看，矢量合成模块需要的电压矢量的模长|Us|就求解出来了。求解角度 θ 需要用到的超前角 θc 也求解出来了。这样的话，矢量合成模块部分的内容就讲完了。而整个带传感器的 FOC 控制对应的框图中涉及到的内容，我们也就全部讲完了。

下面呢，我们需要把之前分析的函数模块汇总起来，实现有传感器 FOC 控制，带动电机旋转。此时呢，我们需要知道程序运行的逻辑，我们程序中，用了 PWM 触发 ADC 转换。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



红框中的代码电机是可以运行的，但要接上霍尔信号。当 ADC 转换完成后，会进入 ADC 中断，电机控制的任务，都放在中断中执行。

```
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    CurrentLoop();
    StatusControl();
    CalSinCos(tFoc.Eangle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
```

所以，大家看下，进入 AD 中断之后，读取转换的结果。然后，根据采样值的，求取三相相电流，再根据三相相电流做一个 Clarke 变换，得到 Iα，Iβ。

```
//获取相电流及Clarke变换之后的Ialpha Ibeta
/*==========*/
void GetCurrent(void)
{
    GetIabc();
    GetIalphabeta();
}
```

然后，为了得到 Id，Iq 电流，还需要进行 Park 变换。

```
//Park变换
//Id = Ialpha * Cos角度 + Ibeta * Sin角度
//Iq = -Ialpha * Sin角度 + Ibeta * Cos角度

static void GetIdq(void)
{
    tIm.qId = MulFrac(tIm.qIalpha,tCore.qCos) + MulFrac(tIm.qIbeta,tCore.qSin);
    tIm.qIq = MulFrac(-tIm.qIalpha,tCore.qSin) + MulFrac(tIm.qIbeta,tCore.qCos);
}

void CurrentLoop(void)
{
    GetIdq();

    tIm.qIdGiven = 0;
    tPiId.sErr = tIm.qIdGiven - tIm.qId;
    tPiId.Kp = 8192;
    tPiId.Ki = tPiId.Kp >> 6;
    tPiId.sLimit = 32767;
    IpiControl(&tPiId);
    tIm.qUd = tPiId.sOut;

    tPiIq.sErr = tIm.qIqGiven - tIm.qIq;
    tPiIq.Kp = 8192;
    tPiIq.Ki = tPiId.Kp >> 6;
    tPiIq.sLimit = 32767;
    IpiControl(&tPiIq);
    tIm.qUq = tPiIq.sOut;
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

得到了 Id，Iq 之后呢，要进行电流闭环控制。控制之后，就得到 Ud，Uq。得到 Ud，Uq 之后呢，要执行矢量合成，目标呢，是要求取超前角跟电压矢量模长。

```
/*=============================================================================*/
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    CurrentLoop();
    StatusControl();
    CalSinCos(tFoc.Eangle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
```

求解矢量模长及超前角，是在状态机中工作状态实现的。

```
/*=============================================================================*/
//状态控制函数
void StatusControl(void)
{
    if(tFoc.DownCount)
        tFoc.DownCount --;

    if(tFoc.Staus == STOP_STATE) //0:Stop Status
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
    }
    else if(tFoc.Staus == BOOTCHARGEINIT_STATE) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tFoc.DownCount = 2000;
        tFoc.Staus = WAITBOOTCHARGE_STATE;
    }
```

我们呢，也再来回顾一下，停止状态，也就没有 PWM 输出。当我们拧电位器旋转，状态就会跳转到自举电容充电初始化状态。在这个状态呢，三个上管是关闭的，三个下管输出 PWM，为给自举电容充电做准备。

```
    else if(tFoc.Staus == BOOTCHARGEINIT_STATE) //1:BootCharge Init
    {
        IOCON1 = IOCON_HINVALID_LPWM;
        IOCON2 = IOCON_HINVALID_LPWM;
        IOCON3 = IOCON_HINVALID_LPWM;
        tFoc.DownCount = 2000;
        tFoc.Staus = WAITBOOTCHARGE_STATE;
    }
    else if(tFoc.Staus == WAITBOOTCHARGE_STATE)//2:BootCharge Finish
    {
        if(tFoc.DownCount == 0)
        {
            IOCON1 = IOCON_HL_INVALID;
            IOCON2 = IOCON_HL_INVALID;
            IOCON3 = IOCON_HL_INVALID;
            HallInitialize();
```

接着呢，状态机会跳转到等待充电完成状态。如果充电完成，就会进入 if 分支，此时会关闭六个管子。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else if(tFoc.Staus == WAITBOOTCHARGE_STATE)//2:BootCharge Finish
{
    if(tFoc.DownCount == 0)
    {
        IOCON1 = IOCON_HL_INVALID;
        IOCON2 = IOCON_HL_INVALID;
        IOCON3 = IOCON_HL_INVALID;
        HallInitialize();

        tPiIq.qSumErr32.S32 = 0;
        tPiId.qSumErr32.S32 = 0;

        if(tFoc.Direction == CCW)
            tIm.qIqGiven = 1024;
        else
            tIm.qIqGiven = -1024;

        tFoc.Staus = WORK_STATE;
    }
}
```

然后，会完成一些初始设置。例如，读取霍尔值，确定初始角度，清零 PI 控制的积分累加值，设定 Iq 电流。

```
else if(tFoc.Staus == WORK_STATE) //3:Run
{
    IOCON1 = IOCON_HPWM_LPWM;
    IOCON2 = IOCON_HPWM_LPWM;
    IOCON3 = IOCON_HPWM_LPWM;

    GetRotorInfo();
    GetUs();
    if(tFoc.Direction == CCW)
        tCore.qEAngle = tFoc.Eangle_Rotor + EANGLE90 + tUm.qDQAngle;
    else
        tCore.qEAngle = tFoc.Eangle_Rotor - EANGLE90 - tUm.qDQAngle;
    tUm.Percent = tUm.PercentOut;
    UsOut();
}
```

然后呢，就会跳转到工作状态，在工作状态呢，三个桥臂驱动波形是互补 PWM 波。然后，要调用获取转子位置角的函数，根据霍尔值求取转子位置角，然后，调用 GetUs()函数，求解|Us|跟超前角。

```
    GetRotorInfo();
    GetUs();
    if(tFoc.Direction == CCW)
        tCore.qEAngle = tFoc.Eangle_Rotor + EANGLE90 + tUm.qDQAngle;
    else
        tCore.qEAngle = tFoc.Eangle_Rotor - EANGLE90 - tUm.qDQAngle;
    tUm.Percent = tUm.PercentOut;
    UsOut();
}
```

根据超前角，再根据转子位置角就可以求解出电压合成矢量夹角，这样的话，Svpwm 模块需要的　两个输入量　θ 跟|Us|就求解出来了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    GetRotorInfo();
    GetUs();
    if(tFoc.Direction == CCW)
        tCore.qEAngle = tFoc.Eangle_Rotor + EANGLE90 + tUm.qDQAngle;
    else
        tCore.qEAngle = tFoc.Eangle_Rotor - EANGLE90 - tUm.qDQAngle;
    tUm.Percent = tUm.PercentOut;
    UsOut();
}
```

求解出来之后，调用 SVPWM 模块的函数，就可以得到 SVPWM 调制波。

```
void UsOut(void)
{
    CalSinCos(tCore.qEAngle);
    GetUsSector(tCore.qEAngle);
    GetUxyz();
    SvpwmOut();
}
```

这样的话，就形成了整个闭环，也就实现了带霍尔传感器的 FOC 控制。

```
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    CurrentLoop();
    StatusControl();
    CalSinCos(tFoc.Eangle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
```

接着，因为 Park 变换需要用到转子位置角。所以，更新了转子位置角之后，我们就查一次表，把对应的正弦值，余弦值计算出来。这样，后面计算 Park 变换就可以用上了。

```
//50us Interrupt
void __attribute__((section(".main_section"),interrupt,shadow,no_auto_psv)) _AD1Interrupt(void)
{
    IFS0bits.AD1IF = 0;

    AdcSample();
    GetCurrent();
    CurrentLoop();
    StatusControl();
    CalSinCos(tFoc.Eangle_Rotor);
    AdcFilter();
    SpiTransmit();
    DownCounter();
}
```

然后，这三个函数呢，分别是对模拟信号滤波，SPI 传送数据到 DAC，还有一个软件递减计数器。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

以上呢，就是整个程序的逻辑。那么，关于有霍尔传感器的 FOC 控制代码，我们就先分析到这里了。那从下次课我们就进入无感 FOC 部分了。我们仍然会先分析无感估算角度的思路，然后再分析程序。

今天我们就先到这里哈，大家早点休息，谢谢大家，晚安！

问：这里 C 调用的函数，形参怎么和汇编的对应起来？第一个形参和第二个形参排序有讲究吗？

答：汇编那边的顺序是 W0,W1,W2,W3，第 1 个形参是 W0， 第 2 个是 W1。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。